

Docker MCP Gateway - CVE-2026-55887

Injection d'arguments via métadonnées de label OCI

Language: Français

Date: 2026-06-28

Author: Security Lab

Classification: Defensive security research and local laboratory documentation

Sommaire

Résumé	3
Points clés	3
Cause racine technique	4
Impact	4
Architecture du lab	5
Preuves des scénarios	5
Scénario vulnérable	5
Scénario corrigé	5
Correction upstream	6
Recommandations défensives	6
Références	6

Résumé

CVE-2026-55887 est une faille d'injection d'arguments de sévérité élevée dans Docker MCP Gateway / Docker MCP Plugin. Elle affecte les versions à partir de 0.21.0 jusqu'aux versions strictement inférieures à 0.42.2. La faille passe par une métadonnée contrôlée par l'attaquant dans le label OCI `io.docker.server.metadata`. Dans les versions vulnérables, ce label est analysé dans une structure de configuration serveur trop large, qui contient aussi des champs capables de modifier le runtime du conteneur.

Ce rapport documente la faille, sa cause racine technique, l'architecture du lab local, la chaîne d'exploitation reproduite dans le périmètre isolé du lab, le correctif upstream introduit en 0.42.2 et les signaux défensifs qui rendent ce comportement observable.

Périmètre

Le contenu est limité à la recherche défensive et à la reproduction locale. Le lab utilise un daemon victime isolé en Docker-in-Docker. Il ne publie pas d'image malveillante sur un registre public et n'expose pas le vrai socket Docker de l'hôte à la gateway vulnérable.

Points clés

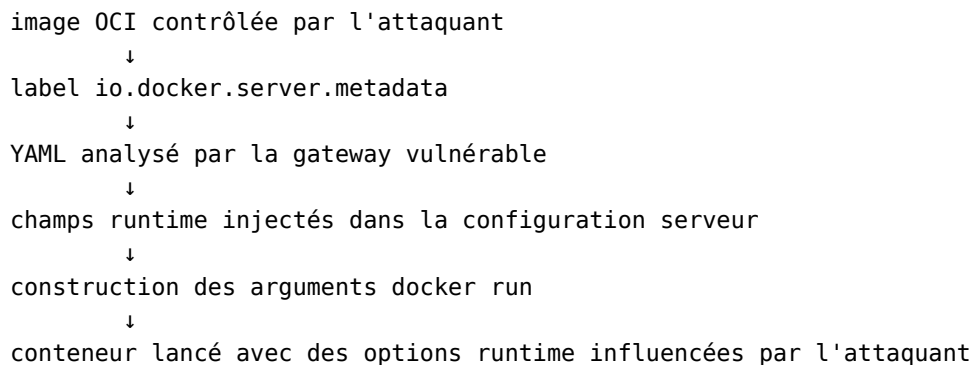
CVE	CVE-2026-55887
Advisory GitHub	GHSA-r2xf-7jw5-pjg6
Advisory ZDI	ZDI-26-363, ZDI-CAN-29539
Package affecté	github.com/docker/mcp-gateway
Versions affectées	<code>>= 0.21.0, < 0.42.2</code>
Version corrigée	0.42.2
Faiblesse	CWE-88: neutralisation incorrecte de délimiteurs d'arguments
Sévérité rapportée	Élevée
Scores rapportés	GitHub CVSS v4.0 8.7; GitLab CVSS v3.1 8.6; ZDI CVSS v3.0 8.6

Table 1: Synthèse des faits issus des bulletins publics.

Cause racine technique

Docker MCP Gateway prend en charge des définitions de serveurs MCP portées par des métadonnées d'images OCI. Dans les versions vulnérables, la gateway analyse le label `io.docker.server.metadata` comme du YAML dans une structure de configuration serveur trop large. Cette structure contient des champs descriptifs, mais aussi des champs qui modifient le comportement runtime du conteneur.

La faille ressemble à une affectation de masse: des métadonnées non fiables, contrôlées par le publisher de l'image, remplissent des champs réservés à une configuration de gateway fiable. Ces champs influencent ensuite le vecteur d'arguments Docker construit par la gateway.



Il s'agit d'une injection d'arguments, pas d'une injection shell classique. Les valeurs dangereuses sont insérées dans le vecteur d'arguments transmis à Docker, et non dans une chaîne de commande interprétée par un shell.

Impact

Les bulletins publics décrivent des scénarios où des métadonnées contrôlées par l'attaquant provoquent le démarrage d'un conteneur MCP avec des options Docker dangereuses. Les exemples incluent des montages du système de fichiers hôte, une exécution avec l'UID 0, ou l'exposition du socket Docker au conteneur lancé.

La frontière de confiance est contournée au moment de la création du conteneur. Les protections comme `no-new-privileges` ne neutralisent pas cette faille, car les options dangereuses sont appliquées avant même que du code s'exécute dans le conteneur.

Dans un environnement réel, une exploitation réussie mène à une compromission de l'hôte. Dans ce lab, le rôle de l'hôte est volontairement remplacé par un daemon victime jetable en Docker-in-Docker.

Architecture du lab

L'architecture technique du lab est portée par le répertoire `lab/`. Ce répertoire sépare les responsabilités de l'environnement: cycle de vie, daemon Docker victime, registre local, gateway réelle, images MCP, observation et poste attaquant. Docker Compose assemble ensuite ces dossiers en services runtime.

```
lab/
|-- scripts/      # cycle de vie, scénarios, preuves et reset
|-- victim/       # daemon Docker-in-Docker représentant l'hôte victime
|-- registry/     # registre OCI local local-registry:5000
|-- gateway/      # vrais builds docker/mcp-gateway v0.42.1 et v0.42.2
|-- images/       # images MCP bénignes et challenge
|-- observer/     # collecte des événements, logs et inspections
|-- attacker/     # poste local de construction et inspection d'images
```

Le dossier `lab/gateway/` est central: il ne contient pas une imitation, mais les définitions permettant de construire et lancer le vrai Docker MCP Gateway upstream. Le parcours vulnérable utilise `v0.42.1`; le parcours corrigé utilise `v0.42.2`.

Au runtime, `lab/victim/` devient le service `victim-docker`, `lab/registry/` devient `local-registry`, `lab/gateway/` produit `vulnerable-gateway` et `fixed-gateway`, `lab/observer/` devient `observer`, et `lab/attacker/` devient `attacker-workstation`. Les images définies dans `lab/images/` sont poussées dans `local-registry:5000`, puis consommées par la gateway via le flux `docker://local-registry:5000/...`

La gateway parle au daemon Docker du lab via `DOCKER_HOST=tcp://victim-docker:2375`. Elle ne monte pas `/var/run/docker.sock` depuis le vrai hôte. Le registre local garde les images challenge à l'intérieur du périmètre du lab, et l'impact reste limité au daemon Docker-in-Docker jetable.

Preuves des scénarios

Scénario vulnérable

Le scénario vulnérable utilise Docker MCP Gateway `v0.42.1`, qui appartient à l'intervalle de versions affectées. L'image challenge contient des métadonnées OCI piégées. Lorsque la gateway traite cette image, les champs qui modifient le runtime rejoignent le chemin de lancement du conteneur et apparaissent dans les preuves Docker collectées depuis `victim-docker`.

Les preuves du scénario portent sur:

- le contenu du label de l'image;
- les logs de gateway montrant le traitement de l'image;
- les événements Docker du daemon victime isolé;
- les montages et la configuration utilisateur du conteneur lancé;
- la preuve que la cible est le faux hôte victime, pas le vrai hôte.

Scénario corrigé

Le scénario corrigé utilise Docker MCP Gateway `v0.42.2`. La même image challenge est traitée via le chemin d'import patché. Les champs runtime issus du label ne produisent plus le comportement Docker runtime vulnérable.

La comparaison porte sur:

- les logs de gateway entre `v0.42.1` et `v0.42.2`;
- les événements Docker des deux exécutions;

- les montages et la configuration utilisateur du conteneur;
- l'accès aux données du faux hôte;
- le traitement des champs runtime suspects.

Correction upstream

La version 0.42.2 modifie la manière dont les métadonnées importées depuis les labels OCI sont analysées et considérées comme fiables.

Le patch introduit `catalog.ImportedServer`, un schéma d'import plus restreint pour les métadonnées de labels d'images. Ce schéma conserve les métadonnées descriptives tout en excluant les champs qui modifient le runtime, comme la commande, les volumes, l'utilisateur, les entrées `extraHosts`, les secrets, les politiques, les paramètres `remote` et la configuration runtime de conteneur associée aux outils. Les déclarations d'environnement importées conservent uniquement les noms, pas les valeurs. Les définitions d'outils importées n'incluent pas de configuration runtime de conteneur.

Le patch supprime aussi l'ancien chemin `self-contained docker://`, qui chargeait les métadonnées du label directement dans une définition serveur complète. Les données serveur issues d'OCI passent désormais par le chemin d'import restreint. En défense en profondeur, les valeurs ajoutées après les flags Docker `-v`, `-u` et `--add-host` sont vérifiées; les valeurs commençant par `-` sont ignorées afin d'éviter qu'une valeur avec tiret initial devienne une option Docker supplémentaire.

Recommandations défensives

La correction principale consiste à mettre à jour Docker MCP Gateway en version 0.42.2 ou supérieure.

Les contrôles opérationnels incluent:

- traiter les images de serveurs MCP comme du logiciel exécutable;
- n'autoriser que des publishers et registres MCP de confiance;
- épinglez les images approuvées par digest;
- éviter les montages du socket Docker dans les conteneurs MCP;
- éviter les montages larges du système de fichiers hôte;
- isoler les charges MCP expérimentales;
- surveiller les événements Docker Engine pour repérer les montages, utilisateurs, surcharges de commande et arguments de création de conteneur inattendus.

Références

- Docker / GitHub Security Advisory GHSA-r2xf-7jw5-pjg6. Bulletin upstream principal pour les versions affectées, la sévérité, l'impact et la correction.
<https://github.com/docker/mcp-gateway/security/advisories/GHSA-r2xf-7jw5-pjg6>
- GitLab Advisory Database CVE-2026-55887. Entrée indépendante pour le package, l'intervalle de versions, le vecteur CVSS v3.1 et les références.
<https://advisories.gitlab.com/golang/github.com/docker/mcp-gateway/CVE-2026-55887/>
- Trend Micro Zero Day Initiative ZDI-26-363. Publication coordonnée avec score CVSS v3.0, chronologie et conditions d'exploitation.
<https://www.zerodayinitiative.com/advisories/ZDI-26-363/>
- Diff upstream v0.42.1...v0.42.2. Diff source utilisé pour vérifier les détails de correction.
<https://github.com/docker/mcp-gateway/compare/v0.42.1...v0.42.2>

- Schéma d'import corrigé. Schéma `catalog.ImportedServer` qui restreint les imports depuis labels OCI.
https://github.com/docker/mcp-gateway/blob/v0.42.2/pkg/catalog/import_spec.go
- Chemin de parsing OCI corrigé. Code patché convertissant les métadonnées de label en définition serveur restreinte.
<https://github.com/docker/mcp-gateway/blob/v0.42.2/pkg/workingset/workingset.go>
- Garde corrigée sur les arguments Docker. Vérifications en défense en profondeur sur les valeurs passées aux flags Docker.
<https://github.com/docker/mcp-gateway/blob/v0.42.2/pkg/gateway/clientpool.go>
- MITRE CWE-88. Classification de faiblesse liée à la neutralisation incorrecte de délimiteurs d'arguments.
<https://cwe.mitre.org/data/definitions/88.html>